

NovaCart Account Dashboard Requirements Document

Project	NovaCart Account Dashboard
Client	NovaCart (30+ country online retailer)
Timeline	5 days (Day 1-5)
Date	August 2026
Version	1.0 (Corrected per Lab Guide)

1. Executive Summary

This document defines the requirements for the NovaCart Account Dashboard, a web application that enables account managers across 30+ countries to access clean sales data without waiting for weekly email reports. The application consists of a FastAPI backend querying Snowflake Gold tables and a React frontend presenting data in three views: Orders Overview, Product Performance, and Customer List. The solution will be deployed on Snowpark Container Services (SPCS) and must be fully functional, documented, and presentable within 5 days.

2. Business Need

2.1 Problem Statement

NovaCart's Data Engineering team has successfully built a reliable ETL pipeline, delivering clean, trusted data in four Gold tables (fact_orders, dim_customer, dim_product, dim_date). However, account managers still rely on ad-hoc Excel reports emailed weekly by the analytics team. These reports are always at least one week stale, numbers often don't match, and AMs must send Slack messages and wait for answers when they need specific insights.

2.2 Desired State

Account managers should be able to:

- View up-to-date order, customer, and product performance data on demand
- Analyze trends over custom time periods without manual requests
- Identify top-performing products and customers instantly
- See geographic revenue distribution across their franchise
- Access the system from any browser without special tools or training

3. Scope

3.1 In Scope

The following capabilities are included in the MVP:

- **Frontend Views**
 - View 1: Orders Overview (monthly trend + country breakdown)
 - View 2: Product Performance (top 10 products by revenue)
 - View 3: Customer List (top 20 customers by revenue)
- **Filters**
 - Date range picker (start date, end date) with Apply button
 - Same filter across all 3 views for consistency

- **API Endpoints (5 total)**

- GET /franchise/{id}/summary — total revenue, orders, active customers
- GET /franchise/{id}/orders — monthly revenue/volume trend
- GET /franchise/{id}/products — top 10 products by revenue
- GET /franchise/{id}/customers — top 20 customers by revenue
- GET /franchise/{id}/countries — revenue grouped by country

- **Charts & Visualizations**

- Bar and line charts for trend analysis
- Sortable tables for product and customer lists
- Stat cards for key metrics (total revenue, total orders)

- **Technical Features**

- FastAPI backend with Python 3.11+
- React 18 frontend
- Snowflake connection via OAuth (SPCS-managed)
- Swagger documentation at /docs
- Error handling with JSON responses
- Loading states in UI
- Dark mode toggle
- Service status indicator in navbar
- SPCS deployment (Docker + NGINX)

3.2 Out of Scope

The following features are explicitly excluded from the MVP to manage timeline and complexity. They may be considered for Phase 2:

- ❌ Country or product line as selectable filters (franchise view is global)
- ❌ Real-time streaming or live data updates (daily refresh is sufficient)
- ❌ Custom date pickers beyond start/end selection
- ❌ Export to Excel, PDF, or CSV
- ❌ Email alerts or scheduled reports
- ❌ Mobile app (desktop/tablet browsers only)
- ❌ Multi-language support (English only)
- ❌ User accounts, roles, or permissions (all users see all franchise data)
- ❌ Customer detail drill-down pages
- ❌ Saved views or bookmarked filters
- ❌ AI/ML features (predictive analytics, forecasting)
- ❌ Data entry or order creation workflows

4. Functional Requirements

4.1 View 1: Orders Overview

Purpose: Show order and revenue trends over time plus geographic distribution

Components:

- Stat cards at top showing Total Revenue and Total Orders for selected date range
- Bar or line chart displaying monthly revenue over time
- Bar chart showing revenue grouped by country
- Date range filter (start date, end date) with Apply button

API Endpoints Used:

- GET /franchise/{id}/summary → powers stat cards
- GET /franchise/{id}/orders?start=YYYY-MM-DD&end=YYYY-MM-DD → monthly trend chart
- GET /franchise/{id}/countries?start=YYYY-MM-DD&end=YYYY-MM-DD → country breakdown

Acceptance Criteria:

- ☐ Stat cards display correct totals for selected date range
- ☐ Monthly chart renders with labeled axes and correct data points
- ☐ Country chart shows all countries with revenue > 0 in the period
- ☐ Applying a new date range updates all three components
- ☐ Loading states visible during API calls
- ☐ Errors handled gracefully with user-friendly messages

4.2 View 2: Product Performance

Purpose: Identify which products are driving revenue

Components:

- Bar chart showing top 10 products by revenue
- Table with columns: Product Name, Category, Units Sold, Revenue
- Table rows are sortable by any column
- Date range filter (same as View 1)

API Endpoint Used:

- GET /franchise/{id}/products?start=YYYY-MM-DD&end=YYYY-MM-DD

Acceptance Criteria:

- ☐ Bar chart displays exactly 10 products (or fewer if < 10 exist)
- ☐ Table rows match the chart data
- ☐ Clicking column headers re-sorts the table
- ☐ Date filter updates both chart and table

- ☐ Revenue values formatted as currency (e.g., \$12,345.67)
- ☐ No broken product names or missing categories

4.3 View 3: Customer List

Purpose: Show which customers are most valuable to the franchise

Components:

- Table with columns: Customer Name, Country, Order Count, Total Spent
- Shows top 20 customers by revenue
- Rows sortable by any column
- Date range filter (same as View 1 and 2)

API Endpoint Used:

- GET /franchise/{id}/customers?start=YYYY-MM-DD&end=YYYY-MM-DD

Acceptance Criteria:

- ☐ Table displays exactly 20 customers (or fewer if < 20 exist)
- ☐ Total Spent column formatted as currency
- ☐ Order Count is an integer (no decimals)
- ☐ Country names are correct and not truncated
- ☐ Sorting works on all 4 columns
- ☐ Date filter updates the customer list

5. API Specification

All API endpoints must be built with FastAPI and connect to Snowflake Gold tables. Swagger documentation must be available at /docs.

GET /franchise/{franchise_id}/summary

Description: Returns a summary of the franchise.

Parameters: franchise_id (path parameter)

Sample Response:

```
{
  "total_revenue": 2450000.00,
  "total_orders": 54321,
  "active_customers": 8901,
  "date_range": {
    "start": "2023-01-01",
    "end": "2023-12-31"
  }
}
```

GET /franchise/{franchise_id}/orders

Description: Returns order volume and revenue grouped by month for the specified date range.

Parameters: franchise_id (path), start (YYYY-MM-DD), end (YYYY-MM-DD)

Sample Response:

```
{
  "data": [
    {"month": "2023-07", "revenue": 185000.00, "order_count": 4200},
    {"month": "2023-08", "revenue": 192000.00, "order_count": 4350},
    {"month": "2023-09", "revenue": 201000.00, "order_count": 4520}
  ]
}
```

GET /franchise/{franchise_id}/products

Description: Returns the top 10 products by revenue for the franchise in the specified date range.

Parameters: franchise_id (path), start (YYYY-MM-DD), end (YYYY-MM-DD)

Sample Response:

```
{
  "products": [
    {
      "product_id": "P-1234",
      "name": "Diagnostics Kit Pro",
      "category": "Diagnostics",

```

```

        "units_sold": 2100,
        "revenue": 525000.00
    }
]
}

```

GET /franchise/{franchise_id}/customers

Description: Returns the top 20 customers by revenue for the franchise in the specified date range.

Parameters: franchise_id (path), start (YYYY-MM-DD), end (YYYY-MM-DD)

Sample Response:

```

{
  "customers": [
    {
      "customer_id": "C-5678",
      "name": "MedSupply Canada",
      "country": "Canada",
      "order_count": 150,
      "total_spent": 1200000.00
    }
  ]
}

```

GET /franchise/{franchise_id}/countries

Description: Returns revenue grouped by country for the franchise in the specified date range.

Parameters: franchise_id (path), start (YYYY-MM-DD), end (YYYY-MM-DD)

Sample Response:

```

{
  "countries": [
    {"country": "United States", "revenue": 850000.00},
    {"country": "Canada", "revenue": 420000.00},
    {"country": "Brazil", "revenue": 310000.00}
  ]
}

```


6. Non-Functional Requirements

Category	Requirement
Performance	Each view must load in < 2 seconds with real Snowflake data
Availability	99% uptime during business hours (8 AM - 6 PM EST, Mon-Fri)
Browser Support	Chrome, Firefox, Safari, Edge (latest 2 versions)
Accessibility	Readable color contrast, keyboard navigable, screen-reader friendly labels
Security	OAuth via SPCS, no hardcoded credentials, HTTPS only
Documentation	API documented at /docs, README in repo with setup instructions

7. Constraints & Assumptions

7.1 Constraints

- Timeline: 5 days from start to final presentation
- Platform: Must deploy on SPCS (Snowpark Container Services)
- Data: Must use provided Snowflake Gold tables (cannot modify schema)
- Team: App Developer (builds code) + App Consultant (validates, documents, presents)
- Infrastructure: Facilitator manages Snowflake account, compute, and service creation

7.2 Assumptions

- Data is refreshed daily in Snowflake (no need for real-time streaming)
- All users have access to all franchise data (no row-level security needed)
- Franchise ID is provided by the facilitator (not user-selected)
- Date range defaults to last 90 days if not specified
- Revenue values are already in USD (no currency conversion needed)
- Product categories exist and are clean in dim_product table
- Customer names are safe to display (no PII concerns flagged)

8. Risks & Mitigation

Risk	Impact	Mitigation
Snowflake query performance degrades with large date ranges	High	Limit date range to max 1 year, add query timeout
API returns 500 errors during demo	High	Test all endpoints thoroughly on Day 4, have fallback mock data

SPCS deployment fails or is delayed	Critical	Test deployment on Day 3, have local demo ready as backup
Charts render incorrectly or data does not display	Medium	Use known charting library (Recharts), validate with sample data first
Date filter produces no results	Low	Add validation: if no data, show friendly message instead of empty chart
Scope creep requests during client role-play	Medium	Reference this requirements doc, defer to Phase 2

9. Acceptance Criteria

The solution is considered complete when all of the following are verified:

API

- ☐ FastAPI with Python 3.11+
- ☐ Snowflake connection via environment variables
- ☐ SPCS OAuth via /snowflake/session/token
- ☐ Swagger docs at /docs
- ☐ /health endpoint working
- ☐ All errors return JSON with message and status code
- ☐ GET /franchise/{id}/summary working
- ☐ GET /franchise/{id}/orders working
- ☐ GET /franchise/{id}/products working
- ☐ GET /franchise/{id}/customers working
- ☐ GET /franchise/{id}/countries working

Frontend

- ☐ React 18
- ☐ All API calls via /api/*
- ☐ SPCS OAuth flow working
- ☐ Loading states and error handling
- ☐ Navbar with service status indicator
- ☐ Dark mode toggle
- ☐ View 1: Orders Overview complete
- ☐ View 2: Product Performance complete
- ☐ View 3: Customer List complete
- ☐ Date range filter working across all views
- ☐ Charts render correctly with real data
- ☐ Tables are sortable

Deployment

- ☐ Backend Docker image built and pushed
- ☐ Frontend Docker image built and pushed
- ☐ NGINX image configured and pushed
- ☐ SPCS services created and running
- ☐ Public URL accessible from any browser

Documentation

- ☐ Requirements document in repo

- ☐ Solution Design Document complete
- ☐ README with setup instructions
- ☐ API documentation at /docs functional

Presentation

- ☐ 10-minute demo prepared
- ☐ Architecture walkthrough ready
- ☐ Trade-offs and decisions documented
- ☐ Q&A responses prepared

10. Success Metrics

This project will be evaluated based on:

- Technical Execution: Does the application work? Are all endpoints functional? Do all views display correctly?
- Code Quality: Is the code clean, documented, and maintainable?
- Usability: Can a non-technical user navigate the dashboard and find answers?
- Architecture: Is the solution well-designed? Are technology choices justified?
- Documentation: Are the requirements and solution design clear and complete?
- Presentation: Can the team explain what they built, why, and what trade-offs were made?
- Scope Management: Did the team deliver a complete MVP without overbuilding?
- Collaboration: Did the App Developer and App Consultant work effectively together?

11. Sign-off

By signing below, all parties acknowledge that this requirements document accurately reflects the scope and deliverables for the NovaCart Account Dashboard project.

Role	Name	Signature & Date
App Developer	[Your Name]	
App Consultant	[Your Name]	
Facilitator	[Facilitator Name]	

Appendix A: Data Model Reference

The application queries four Gold tables in Snowflake:

fact_orders

- order_id: Unique identifier for each order
- customer_id: Foreign key to dim_customer
- product_id: Foreign key to dim_product
- date_key: Foreign key to dim_date
- quantity: Number of units ordered
- amount: Revenue for this order line
- status: Order status (e.g., Completed, Pending)
- order_date: Date the order was placed

dim_customer

- customer_id: Unique identifier
- name: Customer name
- email: Contact email
- city: Customer city
- state: Customer state/province
- is_current: Flag indicating current record (SCD Type 2)

dim_product

- product_id: Unique identifier
- name: Product name
- category: Product category/line
- price: Current list price

dim_date

- date_key: Integer date key (YYYYMMDD)
- full_date: Actual date
- year: Year
- quarter: Quarter (Q1, Q2, Q3, Q4)
- month: Month number (1-12)
- month_name: Month name (January, February, etc.)
- day_of_week: Day of week (Monday, Tuesday, etc.)